

UNLOCK

CHAPTER 23

Policy Gradients & Continuous Action Spaces

From discrete decisions to smooth, continuous control — mastering PPO and policy gradient methods for real-world reinforcement learning.



UNLOCK

OVERVIEW

What We'll Cover

This module bridges the gap between discrete-action RL (like DQN) and the continuous-action world that most real-world control problems demand.

01

The Continuous Action Problem

Why DQN fails when actions aren't discrete choices

03

Proximal Policy Optimization

The industry-standard algorithm for stable policy updates

02

Policy Gradient Methods

Learning the policy directly instead of value functions

04

Implementation with Stable-Baselines3

Hands-on tooling for production-grade RL



UNLOCK

23.1 — THE PROBLEM

The Limits of Discrete Actions

Deep Q-Networks (DQN) excel when an agent chooses from a finite set of actions — "move left," "move right," "jump." But most physical control tasks operate in **continuous action spaces**: steering a car by a precise angle, applying a specific torque to a robotic joint, or adjusting a thermostat to an exact temperature. Discretizing these spaces into bins introduces quantization error and causes the action space to explode combinatorially. Policy gradient methods were developed specifically to handle this challenge by outputting continuous values directly.

Discrete vs. Continuous Actions

Understanding the fundamental difference between these two paradigms is critical before diving into policy gradients.

Discrete Action Spaces

- Finite set of choices (e.g., left, right, up, down)
- DQN computes Q-values for every action
- Agent picks the action with max Q-value
- Works well for Atari games, grid worlds

Continuous Action Spaces

- Infinite possible values (e.g., angle $\in [-30^\circ, +30^\circ]$)
- Cannot enumerate all actions for Q-value lookup
- Agent must output precise real-numbered values
- Required for robotics, autonomous driving, control

Policy Gradient Methods Explained

Instead of learning a value function and deriving a policy from it, **policy gradient methods learn the policy directly**. The policy $\pi_{\theta}(a|s)$ is a parameterized function (typically a neural network) that maps states to a probability distribution over actions. For continuous spaces, this distribution is often a Gaussian where the network outputs the mean μ and standard deviation σ .



The agent samples actions from this distribution, collects trajectories, and then adjusts θ via gradient ascent on expected cumulative reward. This is formalized by the **policy gradient theorem**.

The Policy Gradient Theorem

The core update rule for policy gradient methods follows a deceptively elegant formulation. The gradient of the expected return with respect to policy parameters is:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot R]$$

$$\pi_{\theta}(a|s)$$

The policy network — probability of taking action a in state s

$$\nabla_{\theta} \log \pi_{\theta}$$

The score function — tells us how to shift the distribution

$$R$$

The return signal — scales the update by how good the outcome was

Intuitively: if an action led to high reward, increase its probability. If it led to low reward, decrease it. The log-probability trick makes this gradient tractable.

Why Vanilla Policy Gradients Aren't Enough

While the policy gradient theorem is elegant, naïve implementations — often called REINFORCE — suffer from serious practical issues that limit their usefulness in complex environments.



High Variance

Gradient estimates are noisy because they depend on sampled trajectories. Small batches lead to wildly fluctuating updates.



Sample Inefficiency

Each batch of data is used once then discarded. Thousands of environment interactions may yield a single gradient step.



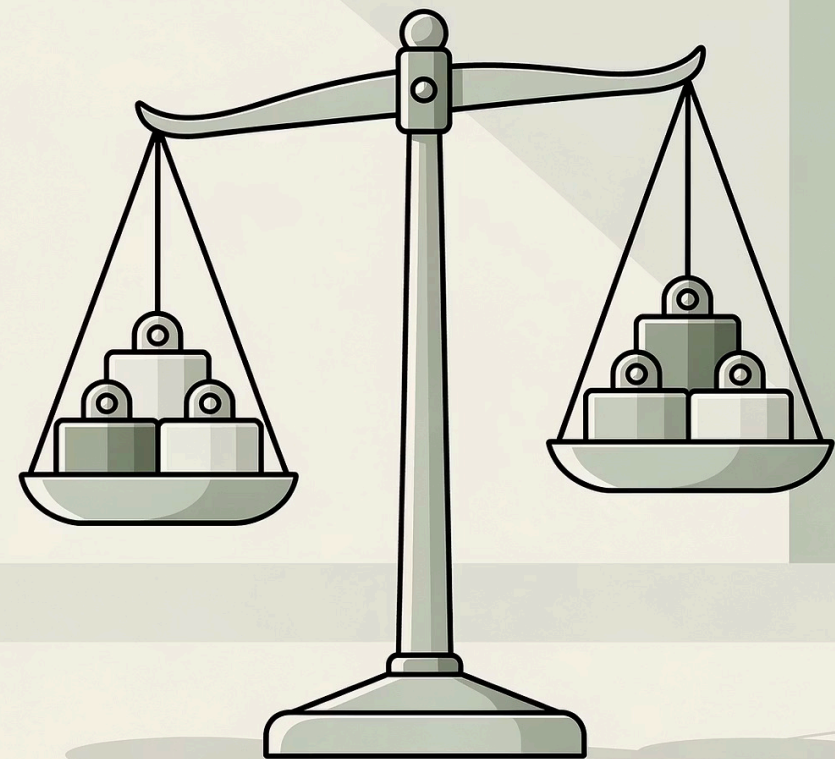
Catastrophic Updates

A single large gradient step can destroy a good policy. There's no built-in mechanism to constrain update size.

These challenges motivated the development of more sophisticated algorithms — most notably, **Proximal Policy Optimization (PPO)**.

Introducing Proximal Policy Optimization

Proximal Policy Optimization (PPO), introduced by OpenAI in 2017, has become the de facto standard for policy gradient training. It is used in everything from robotic locomotion to fine-tuning large language models (RLHF). PPO achieves its remarkable stability by enforcing a simple but powerful constraint: **the new policy must stay close to the old policy**. It does this by clipping the probability ratio between old and new policies, preventing any single update from making a drastic change. The result is an algorithm that is easy to implement, tune, and scale.



How PPO's Clipped Objective Works

PPO's key innovation is its **clipped surrogate objective**. It computes the ratio between the new and old policy probabilities, then clips this ratio to a narrow band around 1.0 (typically ± 0.2).

$$L^{CLIP}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

1

Compute Ratio

Calculate $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ — how much has the policy changed?

2

Clip the Ratio

Restrict r_t to $[1 - \epsilon, 1 + \epsilon]$. With $\epsilon = 0.2$, ratios stay between 0.8 and 1.2.

3

Take the Minimum

Use the **pessimistic** bound — whichever is lower between clipped and unclipped. This prevents overconfident updates.

The advantage estimate $\hat{A}_t$ tells the algorithm whether an action was better or worse than expected, guiding the direction of the update.

Why PPO Dominates Modern RL

PPO's popularity stems from its unique combination of practical strengths that make it suitable for both research and production environments.



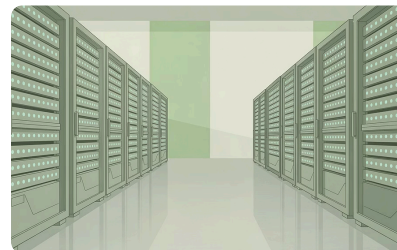
Simple to Implement

No complex second-order optimization or conjugate gradients. The clipped objective is straightforward to code and debug.



Training Stability

The clipping mechanism prevents catastrophic policy collapse. Training curves are smooth and predictable compared to alternatives.



Parallelizable

Data collection and gradient computation scale naturally across multiple workers and environments in parallel.

PPO vs. Other Policy Gradient Methods

Understanding where PPO sits in the landscape of policy optimization helps clarify its design tradeoffs.

Feature	REINFORCE	TRPO	PPO
Update constraint	None	KL divergence (hard)	Clipped ratio (soft)
Implementation	Very simple	Complex (conjugate grad.)	Simple
Stability	Low	High	High
Sample efficiency	Low	Moderate	Moderate-High
Scalability	Good	Limited	Excellent
Popularity	Educational	Research	Industry standard

PPO essentially captures the stability benefits of TRPO (Trust Region Policy Optimization) while being dramatically simpler to implement and more computationally efficient.



UNLOCK

IMPLEMENTATION

Stable-Baselines3: Your RL Toolkit

To implement PPO and other advanced RL algorithms without reinventing the wheel, we use **Stable-Baselines3 (SB3)**. This open-source Python library provides reliable, well-tested implementations of major RL algorithms built on top of PyTorch. SB3 follows a consistent API pattern: define your environment, choose an algorithm, train, and evaluate. It integrates seamlessly with OpenAI Gymnasium environments and supports custom environments, callbacks, logging, and vectorized (parallel) environments out of the box.

PPO in 10 Lines of Code

One of PPO's greatest practical advantages is how quickly you can go from idea to running experiment. Here's a minimal working example using Stable-Baselines3:

```
import gymnasium as gym
from stable_baselines3 import PPO

# Create a continuous-action environment
env = gym.make("Pendulum-v1")

# Initialize PPO with an MLP policy
model = PPO("MlpPolicy", env, verbose=1,
            learning_rate=3e-4, clip_range=0.2)

# Train for 100,000 timesteps
model.learn(total_timesteps=100_000)

# Evaluate the trained agent
obs, _ = env.reset()
for _ in range(1000):
    action, _ = model.predict(obs, deterministic=True)
    obs, reward, done, truncated, info = env.step(action)
    if done or truncated:
        obs, _ = env.reset()
```

The `clip_range=0.2` parameter directly controls the ϵ value in PPO's clipped objective. The `MlpPolicy` creates a feedforward network that outputs Gaussian distribution parameters for continuous actions.

Key Hyperparameters to Tune

PPO is forgiving compared to many RL algorithms, but tuning these hyperparameters can significantly improve performance on your specific task.



clip_range (ϵ)

Controls how far the policy can change per update. Default: 0.2. Lower values (0.1) for more conservative updates; higher (0.3) for faster adaptation.



learning_rate

Step size for gradient updates. Default: 3e-4. Consider using a learning rate schedule that decays over training for better convergence.



n_steps

Number of steps collected before each update. Default: 2048. Larger values reduce variance but slow down iteration speed.



n_epochs

Number of passes over collected data per update. Default: 10. More epochs extract more learning per batch but risk overfitting to stale data.

What You Should Remember

1 Policy gradients learn the policy directly

Instead of computing Q-values for every possible action, policy gradient methods output actions from a parameterized distribution — enabling continuous control.

2 PPO is the go-to algorithm

Its clipped surrogate objective delivers the stability of trust-region methods with the simplicity of vanilla policy gradients. It's the industry default for good reason.

3 Stable-Baselines3 makes it practical

With SB3, you can go from zero to a trained continuous-action agent in minutes. Focus on environment design and hyperparameter tuning, not algorithm internals.

